

机器学习原理与应用

李石峰

2024

目录

第一章 章节习题及参考答案	5
1.1 基本术语-章节习题	5
1.1.1 简答题	5
1.1.2 计算题	7
1.2 模型评估-章节习题	8
1.2.1 简答题	8
1.2.2 计算题	8
1.2.3 编程题	9
1.3 线性模型-章节习题	10
1.3.1 简答题	10
1.3.2 计算题	11
1.3.3 编程题	12
1.4 决策树-章节习题	14
1.4.1 简答题	14
1.4.2 计算题	15
1.5 svm-章节练习	16
1.5.1 简答题	16
1.5.2 计算题	17
1.5.3 编程题	17
1.6 贝叶斯-章节习题	19
1.6.1 简答题	19
1.6.2 计算题	19
1.6.3 编程题	20
1.7 聚类-章节练习	21
1.7.1 简答题	21
1.7.2 计算题	23
1.7.3 编程题	25
1.8 数据降维-章节习题	26
1.8.1 简答题	26
1.8.2 计算题	27
1.8.3 编程题	27
1.9 集成学习-章节练习	28
1.9.1 简答题	28

1.9.2	计算题	28
1.9.3	编程题	29
1.10	特征选择-章节练习	30
1.10.1	简答题	30
1.10.2	计算题	31
1.10.3	编程题	31

第一章 章节习题及参考答案

1.1 基本术语-章节习题

1.1.1 简答题

1. 为什么 Python 是机器学习开发中最常用的编程语言？它有哪些优势？
2. 在机器学习项目中，如何选择合适的运行环境（如本地环境、云端平台或分布式计算框架）？
3. 常见的机器学习库（如 NumPy、Pandas、Scikit-learn、TensorFlow、PyTorch）分别适用于哪些任务？请举例说明。
4. 解释什么是依赖冲突，并描述一种解决依赖冲突的方法。
5. 如何优化程序运行环境以提高代码执行效率？请列举至少三种方法。
6. 什么是交叉验证，它为什么重要？
7. 如何判断训练好的模型是过拟合还是欠拟合？
8. 如何防止模型过拟合或者欠拟合？
9. 模型评估中，如何区分模型的泛化能力和复杂度？
10. 现在要训练一个模型，通过该模型可以预测房价，请设计训练数据对应的特征？

答案：

1. Python 是机器学习开发中最常用的编程语言，主要因为其语法简洁易学、生态系统丰富且社区活跃。此外，Python 拥有大量专为机器学习设计的库（如 NumPy、Pandas、Scikit-learn 等），能够高效处理数据、构建模型并进行可视化。
2. 选择运行环境时需考虑以下因素：
 - 数据规模：小规模数据可在本地环境中处理，而大规模数据可能需要云端或分布式计算框架。
 - 硬件资源：如果本地硬件资源有限，可以选择云端平台（如 Google Colab、AWS）来利用高性能 GPU/TPU。
 - 开发需求：对于初学者或快速原型开发，Jupyter Notebook 和 Google Colab 更方便；对于生产环境，则需要更稳定的本地或分布式环境。
3. 常见机器学习库及其适用任务：

- NumPy: 用于高效的数值计算, 例如矩阵运算。
 - Pandas: 用于数据清洗和预处理, 例如处理缺失值、数据聚合。
 - Scikit-learn: 用于传统机器学习算法 (如线性回归、决策树、K-means 聚类)。
 - TensorFlow/PyTorch: 用于深度学习模型的构建和训练, 例如卷积神经网络 (CNN)、循环神经网络 (RNN)。
4. 依赖冲突是指不同库或工具对同一依赖项的版本要求不一致, 导致程序无法正常运行。解决方法包括:
- 使用虚拟环境 (如 'venv' 或 'conda') 隔离不同项目的依赖。
 - 使用依赖管理工具 (如 'pip-tools' 或 'poetry') 锁定依赖版本。
 - 手动调整依赖版本, 确保兼容性。
5. 优化程序运行环境的方法包括:
- 使用高效的库 (如 NumPy 替代纯 Python 实现)。
 - 并行化处理: 利用多核 CPU 或 GPU 加速计算。
 - 减少冗余依赖: 清理不必要的库以降低环境复杂度。
6. 交叉验证是一种统计分析方法, 用于评估并提高模型的泛化能力。它将数据集分成几个子集, 每个子集轮流作为测试集, 而其余子集作为训练集。这种方法重要是因为它可以帮助我们估计模型在未见过的数据上的性能, 减少过拟合的风险, 并从有限的数据中获取更多信息。
7. 判断模型是过拟合还是欠拟合, 可以通过观察训练和验证损失曲线来判断, 当训练损失持续下降但验证损失开始上升时, 模型可能过拟合; 若训练和验证损失都很高, 则模型可能欠拟合。此外, 观察性能指标, 如果模型在训练集上性能非常好但在验证集上性能较差, 可能是过拟合的迹象; 而模型在训练集和验证集上都表现不佳则可能是欠拟合。还可以通过学习曲线来判断, 欠拟合时训练集和测试集的误差都很高, 且随着训练样本数量的增加, 误差没有明显下降; 过拟合时训练集的误差很低, 但测试集的误差很高, 且随着训练样本数量的增加, 测试集误差逐渐增大。最后, 通过可视化观察, 过拟合时模型在训练集上的预测与实际目标非常吻合, 但在验证集上存在大量错误; 欠拟合时模型无法捕获训练集或验证集中的模式。
8. 防止模型过拟合和欠拟合的方法包括: 对于过拟合, 可以增加数据量、使用数据增强技术、应用正则化 (如 L1、L2 正则化)、在神经网络中使用 Dropout 层、提前停止训练等; 对于欠拟合, 可以增加模型复杂度 (如增加层数、神经元数量等)、减少正则化强度、增加训练轮数等。同时, 在训练过程中合理划分训练集、验证集和测试集, 通过观察验证集的性能来调整模型参数和训练策略, 以达到平衡拟合的效果。
9. 模型的泛化能力指的是模型在未见过的数据上的表现能力, 而复杂度则是指模型的参数数量和模型的拟合程度。一个高泛化能力的模型能够在新数据上保持稳定的性能, 而不会过拟合训练数据。模型复杂度则影响模型的过拟合和欠拟合。通常, 模型复杂度过高可能导致过拟合, 而模型复杂度过低则可能导致欠拟合。模型评估中, 我们通过交叉验证、调整模型参数和正则化等方法来平衡模型的泛化能力和复杂度。
10. 在训练用于判断房价的模型时, 以下特征值得考虑: 房屋的基本信息, 如房屋面积 (以平方米或平方英尺计)、房间数量 (涵盖卧室、客厅、厨房等)、卧室数量、卫生间数量、建造年份 (体现房屋新旧程度)、房屋类型 (如公寓、别墅、排屋等)、房屋朝向 (如朝南、朝北等, 影响采

光和通风)、楼层(房屋所在楼层及总楼层数);地理位置信息,如城市、社区、邻里(具体街区或邻里区域)、距离市中心的距离(以公里计)、附近交通设施(如地铁站、公交站的距离);周边设施,如附近学校数量和质量(包括小学、中学等)、医院数量和距离、商场和购物中心数量及距离、公园和休闲设施数量及距离;房屋状况,如装修状况(毛坯、简装、精装等)、房屋维护状况(是否有损坏、是否需维修等)、是否配备电梯(针对多层建筑);其他特征,如车库数量、花园面积、物业费(如果房屋属于物业管理小区)。这些特征从不同维度反映房屋价值和吸引力,为模型提供多维度学习依据,从而提升预测的准确性和全面性。

1.1.2 计算题

1. 假设一个机器学习任务需要处理的数据集大小为 10GB,而你的本地计算机内存只有 8GB。请设计一种解决方案,确保任务能够顺利完成。
2. 给定以下 Python 代码片段,分析其可能存在的性能瓶颈,并提出优化建议:

```
1 import numpy as np
2
3 # 创建一个大数据组
4 data = np.random.rand(10000, 10000)
5
6 # 计算每列的平均值
7 col_means = [np.mean(data[:, i]) for i in range(data.shape[1])]
8
```

答案:

1. 针对 10GB 数据集超出本地内存的情况,可以采用以下解决方案:
 - 分块处理:将数据分成多个小块,每次加载一块到内存中进行处理,最后合并结果。
 - 使用云端平台:将任务迁移到具有更大内存的云端环境(如 Google Colab 或 AWS EC2 实例)。
 - 数据压缩:对数据进行压缩存储,减少内存占用。
2. 性能瓶颈分析及优化建议:
 - 性能瓶颈:代码使用了 Python 的列表推导式逐列计算平均值,这会导致多次访问数组的列,增加了时间开销。
 - 优化建议:
 - 使用 NumPy 内置函数直接计算列平均值,避免循环:

```
1 col_means = np.mean(data, axis=0)
2
```

- 如果数据过大,无法一次性加载到内存中,可以分块计算每块的列平均值,然后合并结果。

1.2 模型评估-章节习题

1.2.1 简答题

1. 为什么模型评估是机器学习项目中不可或缺的一部分?

2. 在分类问题中，准确率（Accuracy）的局限性是什么？在什么情况下应优先使用精确率（Precision）或召回率（Recall）？
3. 什么是交叉验证？K 折交叉验证的主要优点是什么？
4. 网格搜索和随机搜索的主要区别是什么？它们分别适用于哪些场景？
5. 如何通过 ROC 曲线和 AUC 值评估分类模型的性能？AUC 值为 1 表示什么？

答案：

1. 模型评估是机器学习项目中不可或缺的一部分，因为它能够确保模型的性能、泛化能力和稳定性。通过评估，可以发现模型是否过拟合或欠拟合，并指导后续优化。
2. 准确率的局限性在于它对类别不平衡的数据集不敏感。例如，在正类样本远少于负类样本的情况下，即使模型预测所有样本为负类，准确率也可能很高。在这种情况下，应优先使用精确率（衡量预测正类的准确性）或召回率（衡量正类样本被正确识别的比例）。
3. 交叉验证是一种通过将数据集划分为多个子集来评估模型性能的方法。K 折交叉验证的主要优点是充分利用数据集，减少因数据划分带来的偏差，从而提高模型评估的可靠性。
4. 网格搜索通过对超参数的所有可能组合进行穷举搜索，适合小规模超参数空间；随机搜索则通过随机采样超参数组合，适合大规模超参数空间。网格搜索更精确但计算成本高，而随机搜索效率更高但可能错过最优解。
5. ROC 曲线通过绘制真正类率（TPR）与假正类率（FPR）的关系来评估分类模型的性能。AUC 值表示 ROC 曲线下的面积，AUC 值越接近 1，模型性能越好。AUC 值为 1 表示模型能够完美区分正类和负类。

1.2.2 计算题

1. 给定以下混淆矩阵，计算准确率、精确率、召回率和 F1 分数：

表 1.1: 混淆矩阵		
	预测正类	预测负类
实际正类	50	10
实际负类	5	35

2. 假设一个回归模型的预测值为 [2.1, 3.9, 6.2, 8.0]，真实值为 [2.0, 4.0, 6.0, 8.0]，计算均方误差（MSE）、均方根误差（RMSE）和平均绝对误差（MAE）。
3. 已知某分类模型的 ROC 曲线下的面积（AUC）为 0.85，解释该值的意义并判断模型性能如何。

答案：

1. 根据混淆矩阵：

$$\text{准确率} = \frac{TP + TN}{TP + FP + FN + TN} = \frac{50 + 35}{50 + 10 + 5 + 35} = 0.85$$

$$\text{精确率} = \frac{TP}{TP + FP} = \frac{50}{50 + 5} = 0.909$$

$$\text{召回率} = \frac{TP}{TP + FN} = \frac{50}{50 + 10} = 0.833$$

$$\text{F1 分数} = 2 \cdot \frac{\text{精确率} \cdot \text{召回率}}{\text{精确率} + \text{召回率}} = 2 \cdot \frac{0.909 \cdot 0.833}{0.909 + 0.833} \approx 0.869$$

2. 计算回归指标:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{4} ((2.1 - 2.0)^2 + (3.9 - 4.0)^2 + (6.2 - 6.0)^2 + (8.0 - 8.0)^2) = 0.035$$

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{0.035} \approx 0.187$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| = \frac{1}{4} (|2.1 - 2.0| + |3.9 - 4.0| + |6.2 - 6.0| + |8.0 - 8.0|) = 0.15$$

3. AUC 值为 0.85 表示模型具有较好的区分能力, 能够较好地地区分正类和负类样本。一般来说, AUC 值越接近 1, 模型性能越好; AUC 值为 0.85 表明模型性能良好, 但仍有改进空间。

1.2.3 编程题

1. 使用 Scikit-learn 实现一个分类模型的评估过程, 加载鸢尾花数据集并完成以下任务:

- 划分训练集和测试集。
- 训练逻辑回归模型并计算准确率、精确率、召回率和 F1 分数。

2. 使用 Scikit-learn 实现一个回归模型的评估过程, 加载波士顿房价数据集并完成以下任务:

- 划分训练集和测试集。
- 训练线性回归模型并计算均方误差 (MSE)、均方根误差 (RMSE) 和平均绝对误差 (MAE)。

答案:

1. 示例代码如下:

```

1  from sklearn.datasets import load_iris
2  from sklearn.model_selection import train_test_split
3  from sklearn.linear_model import LogisticRegression
4  from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
5
6  # 加载鸢尾花数据集
7  iris = load_iris()
8  X, y = iris.data, iris.target
9
10 # 划分训练集和测试集
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
12
13 # 训练逻辑回归模型
14 model = LogisticRegression()
15 model.fit(X_train, y_train)
16
17 # 测试模型性能
18 y_pred = model.predict(X_test)
19 accuracy = accuracy_score(y_test, y_pred)
20 precision = precision_score(y_test, y_pred, average='macro')
```

```
21 recall = recall_score(y_test, y_pred, average='macro')
22 f1 = f1_score(y_test, y_pred, average='macro')
23
24 print("准确率:", accuracy)
25 print("精确率:", precision)
26 print("召回率:", recall)
27 print("F1分数:", f1)
28
```

2. 示例代码如下:

```
1 from sklearn.datasets import load_boston
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error, mean_absolute_error
5 import numpy as np
6
7 # 加载波士顿房价数据集
8 boston = load_boston()
9 X, y = boston.data, boston.target
10
11 # 划分训练集和测试集
12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
13
14 # 训练线性回归模型
15 model = LinearRegression()
16 model.fit(X_train, y_train)
17
18 # 测试模型性能
19 y_pred = model.predict(X_test)
20 mse = mean_squared_error(y_test, y_pred)
21 rmse = np.sqrt(mse)
22 mae = mean_absolute_error(y_test, y_pred)
23
24 print("均方误差 (MSE):", mse)
25 print("均方根误差 (RMSE):", rmse)
26 print("平均绝对误差 (MAE):", mae)
27
```

1.3 线性模型-章节习题

为了帮助读者更好地掌握线性模型的核心概念和应用方法,本章提供了以下习题,包括简答题、计算题和编程题。每道题目均附有详细答案。

1.3.1 简答题

1. 什么是线性回归?它与逻辑回归的主要区别是什么?
2. 在线性模型中, L1 正则化和 L2 正则化的作用分别是什么?它们如何影响模型的参数?
3. 什么是梯度下降算法?它在线性模型中的主要作用是什么?
4. 线性模型的假设有哪些?这些假设对模型性能有何影响?
5. 什么是多重共线性问题?如何检测 and 解决多重共线性?

答案：

1. 线性回归是一种用于预测连续目标变量的模型，其目标是通过拟合一条直线（或超平面）来最小化预测值与真实值之间的误差。逻辑回归则用于分类任务，通过 Sigmoid 函数将线性组合映射到概率范围 $[0, 1]$ ，从而实现二分类或多分类。
2. L1 正则化通过对模型参数施加稀疏性约束，将不重要的特征权重置为零，从而实现特征选择；L2 正则化通过限制参数的平方和，使所有特征的权重较小但非零，主要用于防止过拟合。L1 正则化倾向于生成稀疏解，而 L2 正则化倾向于平滑解。
3. 梯度下降是一种优化算法，通过迭代更新模型参数以最小化损失函数。在线性模型中，梯度下降的主要作用是找到最优的模型参数，使得预测值尽可能接近真实值。
4. 线性模型的主要假设包括：
 - 特征与目标变量之间存在线性关系。
 - 数据噪声服从正态分布。
 - 特征之间不存在多重共线性。
 - 残差（预测值与真实值的差异）独立且同分布。

如果这些假设不成立，可能导致模型性能下降或结果不可靠。

5. 多重共线性是指特征之间存在高度相关性，导致模型参数估计不稳定。可以通过方差膨胀因子（VIF）检测多重共线性。解决方法包括移除冗余特征、使用主成分分析（PCA）降维或引入正则化（如 L2 正则化）。

1.3.2 计算题

1. 给定一个简单的线性回归模型 $y = w_1x_1 + w_2x_2 + b$ ，其中 $w_1 = 2, w_2 = -1, b = 3$ 。如果输入数据为 $(x_1, x_2) = (4, 5)$ ，求预测值 y 。
2. 假设一个线性回归模型的目标是最小化均方误差（MSE），给定以下数据集：使用最小二乘法

表 1.2: 线性回归数据集

输入 x	输出 y
1	2.1
2	3.9
3	6.2
4	8.0

计算模型参数 w 和 b 。

3. 给定一个逻辑回归模型，其决策边界为 $z = 2x_1 - 3x_2 + 1$ 。如果输入数据为 $(x_1, x_2) = (1, 1)$ ，求该样本属于正类的概率（假设使用 Sigmoid 函数 $\sigma(z) = \frac{1}{1+e^{-z}}$ ）。

答案：

1. 根据线性回归公式：

$$y = w_1x_1 + w_2x_2 + b = 2 \cdot 4 + (-1) \cdot 5 + 3 = 8 - 5 + 3 = 6$$

因此，预测值为 $y = 6$ 。

2. 最小二乘法的目标是最小化均方误差 (MSE)。首先计算输入矩阵 X 和输出向量 y ：

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}, \quad y = \begin{bmatrix} 2.1 \\ 3.9 \\ 6.2 \\ 8.0 \end{bmatrix}$$

模型参数 w 和 b 的闭式解为：

$$\theta = (X^T X)^{-1} X^T y$$

计算得到：

$$\theta = \begin{bmatrix} b \\ w \end{bmatrix} = \begin{bmatrix} 0.1 \\ 1.95 \end{bmatrix}$$

因此，模型参数为 $b = 0.1$, $w = 1.95$ 。

3. 首先计算决策边界的值 z ：

$$z = 2x_1 - 3x_2 + 1 = 2 \cdot 1 - 3 \cdot 1 + 1 = 0$$

使用 Sigmoid 函数计算概率：

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^0} = 0.5$$

因此，该样本属于正类的概率为 0.5。

1.3.3 编程题

1. 使用 Scikit-learn 实现一个线性回归模型，加载波士顿房价数据集并完成以下任务：

- 划分训练集和测试集。
- 训练线性回归模型并评估其性能（使用均方误差和决定系数 R^2 ）。

2. 使用 Scikit-learn 实现一个逻辑回归模型，加载鸢尾花数据集并完成以下任务：

- 划分训练集和测试集。
- 训练逻辑回归模型并绘制决策边界。

答案：

1. 示例代码如下：

```
1 from sklearn.datasets import load_boston
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error, r2_score
5
6 # 加载波士顿房价数据集
7 boston = load_boston()
```

```
8 X, y = boston.data, boston.target
9
10 # 划分训练集和测试集
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state
    =42)
12
13 # 训练线性回归模型
14 model = LinearRegression()
15 model.fit(X_train, y_train)
16
17 # 测试模型性能
18 y_pred = model.predict(X_test)
19 mse = mean_squared_error(y_test, y_pred)
20 r2 = r2_score(y_test, y_pred)
21 print("均方误差 (MSE):", mse)
22 print("决定系数 (R^2):", r2)
23
```

2. 示例代码如下:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_iris
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LogisticRegression
6
7 # 加载鸢尾花数据集
8 iris = load_iris()
9 X = iris.data[:, :2] # 只使用前两个特征
10 y = iris.target
11 y = y[y != 2] # 只保留两类样本
12 X = X[y != 2]
13
14 # 划分训练集和测试集
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state
    =42)
16
17 # 训练逻辑回归模型
18 model = LogisticRegression()
19 model.fit(X_train, y_train)
20
21 # 绘制决策边界
22 x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
23 y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
24 xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
25 np.arange(y_min, y_max, 0.02))
26 Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
27 Z = Z.reshape(xx.shape)
28
29 plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.Paired)
30 plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o', cmap=plt.cm.Paired)
31 plt.xlabel("Feature 1")
32 plt.ylabel("Feature 2")
33 plt.title("Logistic Regression Decision Boundary")
34 plt.show()
35
```

1.4 决策树-章节习题

1.4.1 简答题

1. 什么是决策树？它在分类和回归任务中的主要区别是什么？
2. 决策树的构建过程中，如何选择最佳特征进行分裂？常用的分裂准则有哪些？
3. 信息增益和基尼指数的主要区别是什么？它们分别适用于哪些场景？
4. 什么是过拟合问题？在决策树中如何通过剪枝技术缓解过拟合？
5. 决策树的优点和缺点分别是什么？在实际应用中需要注意哪些问题？

答案：

1. 决策树是一种基于规则的模型，通过递归分裂数据集来解决分类或回归问题。在分类任务中，目标是预测离散类别（如“是/否”）；而在回归任务中，目标是预测连续值（如房价）。两者的区别在于输出类型不同：分类任务的输出是类别标签，而回归任务的输出是数值。
2. 在决策树的构建过程中，选择最佳特征通常基于分裂准则，目的是找到能够最大程度区分数据的特征。常用的分裂准则包括：
 - 信息增益：衡量使用某个特征后数据集熵的减少量。
 - 基尼指数：衡量数据集的不纯度，基尼指数越小，数据集越纯净。
 - 信息增益比：对信息增益进行归一化处理，避免偏向于取值较多的特征。
3. 信息增益衡量的是熵的减少量，适合处理多分类问题，但对取值较多的特征可能存在偏差。基尼指数衡量的是不纯度，计算效率更高，适合大规模数据集。信息增益更注重减少不确定性，而基尼指数更关注数据集的不纯度。
4. 过拟合是指模型在训练集上表现很好，但在测试集上表现较差的现象。在决策树中，过拟合通常是由于树的深度过大导致模型过于复杂。剪枝技术可以缓解过拟合，包括：
 - 预剪枝：限制树的最大深度、最小样本数等参数，提前停止分裂。
 - 后剪枝：先生成一棵完整的树，然后移除不必要的分支以简化模型。
5. 决策树的优点包括：
 - 直观易懂，可解释性强。
 - 不需要对数据进行复杂的预处理（如特征缩放）。
 - 能够处理非线性关系。

缺点包括：

- 容易过拟合，尤其是当树的深度较大时。
- 对噪声敏感，微小的数据变化可能导致树结构显著不同。
- 偏向于选择取值较多的特征进行分裂。

在实际应用中，需要注意调整超参数（如最大深度）、使用剪枝技术以及结合集成学习方法（如随机森林）来提升性能。

1.4.2 计算题

1. 给定以下数据集，计算基于“天气”特征的信息增益和基尼指数。

表 1.3: 天气与活动数据集

天气	活动
晴天	户外活动
晴天	户外活动
雨天	室内活动
雨天	室内活动
阴天	户外活动
阴天	室内活动

2. 假设一个二分类问题的数据集如下表所示，计算基于“特征 A”的信息增益。

表 1.4: 二分类问题数据集

特征 A	特征 B	类别
1	2	正类
1	3	正类
0	4	负类
0	5	负类
1	6	负类

3. 在一个回归问题中，假设目标变量的真实值为 [3.0, 5.0, 7.0]，预测值为 [2.8, 5.2, 6.9]。计算均方误差 (MSE) 和决定系数 (R^2)。

答案：

1. • 计算原始数据集的熵 $H(D)$ ：

$$H(D) = - \left(\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6} \right) = 1$$

- 计算按“天气”分裂后的加权平均熵：

$$H(D_{\text{晴天}}) = 0, \quad H(D_{\text{雨天}}) = 0, \quad H(D_{\text{阴天}}) = 1$$

$$H(D_{\text{天气}}) = \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 1 = 0.333$$

- 信息增益：

$$\text{Gain}(D, \text{天气}) = H(D) - H(D_{\text{天气}}) = 1 - 0.333 = 0.667$$

- 计算基尼指数：

$$\text{Gini}(D) = 1 - \left(\frac{3^2}{6} + \frac{3^2}{6} \right) = 0.5$$

$$\text{Gini}(D_{\text{天气}}) = \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 0.5 = 0.167$$

2. • 计算原始数据集的熵 $H(D)$:

$$H(D) = -\left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5}\right) \approx 0.971$$

- 计算按“特征 A”分裂后的加权平均熵:

$$H(D_{A=1}) = -\left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3}\right) \approx 0.918$$

$$H(D_{A=0}) = -(0 \cdot \log_2 0 + 1 \cdot \log_2 1) = 0$$

$$H(D_{\text{特征 A}}) = \frac{3}{5} \cdot 0.918 + \frac{2}{5} \cdot 0 = 0.551$$

- 信息增益:

$$\text{Gain}(D, \text{特征 A}) = H(D) - H(D_{\text{特征 A}}) = 0.971 - 0.551 = 0.420$$

3. • 计算均方误差 (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{3} ((3.0 - 2.8)^2 + (5.0 - 5.2)^2 + (7.0 - 6.9)^2) = 0.02$$

- 计算决定系数 (R^2):

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i = \frac{3.0 + 5.0 + 7.0}{3} = 5.0$$

$$\text{SS}_{\text{tot}} = \sum_{i=1}^n (y_i - \bar{y})^2 = (3.0 - 5.0)^2 + (5.0 - 5.0)^2 + (7.0 - 5.0)^2 = 8.0$$

$$\text{SS}_{\text{res}} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = (3.0 - 2.8)^2 + (5.0 - 5.2)^2 + (7.0 - 6.9)^2 = 0.06$$

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{0.06}{8.0} = 0.9925$$

1.5 svm-章节练习

1.5.1 简答题

1. 支持向量机 (SVM) 的基本思想是什么? 它如何处理线性不可分的情况?
2. 什么是核函数? 常用的核函数有哪些?
3. SVM 中的软间隔 (Soft Margin) 有什么作用? 如何通过调整超参数 C 来控制模型的复杂度?

答案:

1. 支持向量机的基本思想是找到一个能够最大化分类边界的超平面, 使得不同类别的样本点尽可能远离该边界。对于线性不可分的情况, SVM 通过引入核函数将数据映射到高维空间, 在高维空间中寻找线性可分的超平面。
2. 核函数是一种将低维空间中的非线性问题转化为高维空间中线性问题的方法。常用的核函数包括:

- 线性核: $K(x_i, x_j) = x_i^T x_j$

- 多项式核: $K(x_i, x_j) = (x_i^T x_j + c)^d$
- 高斯径向基核 (RBF): $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
- Sigmoid 核: $K(x_i, x_j) = \tanh(\alpha x_i^T x_j + c)$

3. 软间隔允许 SVM 在分类时容忍一定的错误分类, 从而避免过拟合。通过调整超参数 C , 可以控制模型的复杂度: 较大的 C 值会惩罚更多的误分类, 导致模型更加复杂; 较小的 C 值则允许更多的误分类, 使模型更加简单。

1.5.2 计算题

1. 给定二维平面上的两个类别样本点: 类别 1 为 $(1, 1), (2, 2)$, 类别 2 为 $(4, 4), (5, 5)$ 。假设使用线性核的 SVM, 请计算最优分类超平面的方程。
2. 如果使用高斯径向基核 (RBF), 且 $\gamma = 1$, 请计算以下两点之间的核函数值: $(0, 0)$ 和 $(1, 1)$ 。

答案:

1. 对于线性可分的情况, SVM 的目标是找到一个超平面 $w^T x + b = 0$, 使得两类样本点之间的间隔最大。根据给定的数据点, 可以计算出超平面的法向量 $w = (1, -1)$ 和偏置 $b = -3$ 。因此, 最优分类超平面的方程为:

$$x_1 - x_2 - 3 = 0$$

2. 高斯径向基核的公式为:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

已知 $\gamma = 1$, 且两点分别为 $(0, 0)$ 和 $(1, 1)$, 计算欧氏距离:

$$\|x_i - x_j\|^2 = (0 - 1)^2 + (0 - 1)^2 = 2$$

因此:

$$K((0, 0), (1, 1)) = \exp(-1 \cdot 2) = \exp(-2) \approx 0.1353$$

1.5.3 编程题

1. 使用 Scikit-learn 实现一个基于 SVM 的二分类模型, 加载鸢尾花数据集并完成以下任务:
 - 划分训练集和测试集。
 - 使用线性核训练 SVM 模型。
 - 计算测试集上的分类准确率。
2. 编写一个 Python 函数, 接收一组二维数据点和标签作为输入, 使用高斯径向基核 (RBF) 训练 SVM 模型, 并绘制决策边界。

答案:

1. 示例代码如下:

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 from sklearn.svm import SVC
4 from sklearn.metrics import accuracy_score
5
```

```

6  # 加载数据集
7  iris = load_iris()
8  X = iris.data[:, :2] # 只使用前两个特征
9  y = iris.target
10 y = y[y != 2] # 只保留两类样本
11 X = X[y != 2]
12
13 # 划分训练集和测试集
14 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
15
16 # 训练SVM模型
17 model = SVC(kernel='linear')
18 model.fit(X_train, y_train)
19
20 # 测试模型性能
21 y_pred = model.predict(X_test)
22 accuracy = accuracy_score(y_test, y_pred)
23 print("测试集准确率:", accuracy)
24

```

2. 示例代码如下:

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.svm import SVC
4
5  def plot_decision_boundary(X, y):
6  # 训练SVM模型
7  model = SVC(kernel='rbf', gamma=1)
8  model.fit(X, y)
9
10 # 创建网格以绘制决策边界
11 x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
12 y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
13 xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
14 np.arange(y_min, y_max, 0.02))
15 Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
16 Z = Z.reshape(xx.shape)
17
18 # 绘制决策边界
19 plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.Paired)
20 plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', marker='o', cmap=plt.cm.Paired)
21 plt.xlabel("Feature 1")
22 plt.ylabel("Feature 2")
23 plt.title("SVM Decision Boundary with RBF Kernel")
24 plt.show()
25
26 # 示例调用
27 X = np.array([[1, 2], [2, 3], [3, 3], [6, 7], [7, 8], [8, 8]])
28 y = np.array([0, 0, 0, 1, 1, 1])
29 plot_decision_boundary(X, y)
30

```

1.6 贝叶斯-章节习题

1.6.1 简答题

1. 解释为什么朴素贝叶斯被称为“朴素”的原因是什么？
2. 在什么情况下朴素贝叶斯可能不是最佳选择？给出至少两个例子。
3. 编写一段 Python 代码实现一个简单的文本分类任务，使用朴素贝叶斯方法。

答案：

1. 朴素贝叶斯被称为“朴素”是因为它假设特征之间是条件独立的（即给定类别的情况下，各个特征之间相互独立）。这一假设在现实中往往不成立，但为了简化计算，朴素贝叶斯模型仍然广泛使用。
2. 朴素贝叶斯可能不是最佳选择的情况包括：
 - 特征之间存在强相关性时，例如图像数据中相邻像素之间通常具有很强的相关性。
 - 数据分布严重偏离朴素贝叶斯的假设（如高斯分布或多项式分布）时，例如非线性复杂数据集。
3. 示例代码如下：

```
1 from sklearn.datasets import fetch_20newsgroups
2 from sklearn.feature_extraction.text import CountVectorizer
3 from sklearn.model_selection import train_test_split
4 from sklearn.naive_bayes import MultinomialNB
5 from sklearn.metrics import accuracy_score
6
7 # 加载数据集
8 categories = ['alt.atheism', 'soc.religion.christian']
9 data = fetch_20newsgroups(subset='all', categories=categories)
10
11 # 文本向量化
12 vectorizer = CountVectorizer()
13 X = vectorizer.fit_transform(data.data)
14 y = data.target
15
16 # 划分训练集和测试集
17 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
18
19 # 训练朴素贝叶斯模型
20 model = MultinomialNB()
21 model.fit(X_train, y_train)
22
23 # 测试模型性能
24 y_pred = model.predict(X_test)
25 print("准确率:", accuracy_score(y_test, y_pred))
26
```

1.6.2 计算题

1. 如果某疾病在人群中的患病率为 1%，而一项检测该疾病的测试有 99% 的准确性（即如果一个人患有此病，测试呈阳性的概率为 99%，且如果一个人不患此病，测试呈阴性的概率也为 99%）。如果一个人的测试结果是阳性，那么他实际上患病的概率是多少？

答案:

1. 根据贝叶斯定理:

$$P(\text{患病}|\text{阳性}) = \frac{P(\text{阳性}|\text{患病}) \cdot P(\text{患病})}{P(\text{阳性})}$$

其中:

$$P(\text{阳性}) = P(\text{阳性}|\text{患病}) \cdot P(\text{患病}) + P(\text{阳性}|\text{未患病}) \cdot P(\text{未患病})$$

已知:

$$P(\text{患病}) = 0.01, \quad P(\text{未患病}) = 0.99$$

$$P(\text{阳性}|\text{患病}) = 0.99, \quad P(\text{阳性}|\text{未患病}) = 0.01$$

代入公式:

$$P(\text{阳性}) = 0.99 \cdot 0.01 + 0.01 \cdot 0.99 = 0.0198$$

因此:

$$P(\text{患病}|\text{阳性}) = \frac{0.99 \cdot 0.01}{0.0198} \approx 0.5$$

即如果一个人的测试结果是阳性, 他实际上患病的概率约为 50%。

1.6.3 编程题

1. 使用提供的训练数据, 构建一个朴素贝叶斯分类器来区分垃圾邮件与非垃圾邮件。
2. 编写一个 Python 函数, 该函数接收一个文本字符串列表和对应的类别标签作为输入, 使用朴素贝叶斯分类器训练模型, 并返回模型的预测准确度。

答案:

1. 示例代码如下:

```

1  from sklearn.feature_extraction.text import CountVectorizer
2  from sklearn.model_selection import train_test_split
3  from sklearn.naive_bayes import MultinomialNB
4  from sklearn.metrics import accuracy_score
5
6  # 假设提供了一个垃圾邮件数据集
7  emails = [
8      "Win money now", "Free entry in a contest", "Hello, how are you?",
9      "Meeting at 5 PM", "Urgent: please reply", "Let's catch up tomorrow"
10 ]
11 labels = [1, 1, 0, 0, 1, 0] # 1 表示垃圾邮件, 0 表示正常邮件
12
13 # 文本向量化
14 vectorizer = CountVectorizer()
15 X = vectorizer.fit_transform(emails)
16
17 # 划分训练集和测试集
18 X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.3,
19 random_state=42)
20
21 # 训练朴素贝叶斯模型
22 model = MultinomialNB()
23 model.fit(X_train, y_train)
24
25 # 测试模型性能

```

```
25 y_pred = model.predict(X_test)
26 print("准确率:", accuracy_score(y_test, y_pred))
27
```

2. 示例代码如下:

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 from sklearn.model_selection import train_test_split
3 from sklearn.naive_bayes import MultinomialNB
4 from sklearn.metrics import accuracy_score
5
6 def train_and_evaluate(texts, labels):
7     # 文本向量化
8     vectorizer = CountVectorizer()
9     X = vectorizer.fit_transform(texts)
10
11     # 划分训练集和测试集
12     X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.3,
13                                                         random_state=42)
14
15     # 训练朴素贝叶斯模型
16     model = MultinomialNB()
17     model.fit(X_train, y_train)
18
19     # 测试模型性能
20     y_pred = model.predict(X_test)
21     accuracy = accuracy_score(y_test, y_pred)
22     return accuracy
23
24 # 示例调用
25 texts = [
26     "Win money now", "Free entry in a contest", "Hello, how are you?",
27     "Meeting at 5 PM", "Urgent: please reply", "Let's catch up tomorrow"
28 ]
29 labels = [1, 1, 0, 0, 1, 0] # 1 表示垃圾邮件, 0 表示正常邮件
30 accuracy = train_and_evaluate(texts, labels)
31 print("模型准确率:", accuracy)
```

1.7 聚类-章节练习

1.7.1 简答题

1. 什么是欧氏距离? 它有哪些特点和应用场景?
2. 曼哈顿距离与欧氏距离的主要区别是什么? 请举例说明其适用场景。
3. 余弦相似度适用于哪些类型的数据? 为什么它不受向量长度的影响?
4. 马氏距离与欧氏距离相比有哪些优势? 它的主要局限性是什么?
5. 杰卡德距离如何定义? 它在文本聚类中有何应用?
6. 简述 K-Means 算法的基本思想及其局限性。
7. DBSCAN 算法中的核心点、边界点和噪声点是如何定义的?

8. 模糊 C 均值 (FCM) 与 K-Means 的主要区别是什么?
9. 层次聚类的优点和缺点分别是什么?
10. 在实际应用中, 如何选择合适的聚类算法?

答案:

1. 欧氏距离是衡量两点之间直线距离的方法, 定义为:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

特点与应用场景:

- 适用于连续型数值数据, 尤其在低维空间中表现良好。
- 对噪声和异常值较为敏感, 需对数据进行归一化或标准化处理。
- 常用于 K-Means 聚类以及层次聚类中的平均链和全链方法。

2. 曼哈顿距离与欧氏距离的主要区别在于:

- 曼哈顿距离是沿坐标轴方向的距离之和, 而欧氏距离是直线距离。
- 曼哈顿距离对噪声和异常值更鲁棒, 适合稀疏数据或高维数据。

应用场景:

- 曼哈顿距离常用于文本分类中的词频向量。
- 欧氏距离常用于 K-Means 聚类。

3. 余弦相似度适用于高维稀疏数据 (如 TF-IDF 向量)。因为它通过计算向量夹角的余弦值来衡量相似性, 而不依赖于向量的模长。因此, 即使两个向量的长度不同, 只要方向一致, 其余弦相似度仍为 1。

4. 马氏距离的优势:

- 考虑了数据的协方差结构, 能够消除维度之间的相关性和尺度差异。
- 适合分布存在异方差性的情况。

局限性:

- 对数据分布假设较强 (需估计协方差矩阵)。
- 计算复杂度较高。

5. 杰卡德距离定义为:

$$d(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

在文本聚类中, 杰卡德距离可用于比较文档词汇集合的相似性。例如, 两篇文档的词汇集合交集越大, 杰卡德距离越小, 表明两篇文档越相似。

6. K-Means 算法的基本思想是将数据集划分为 K 个簇, 使得每个数据点与其所属簇中心的距离最小化。其目标是 minimized 目标函数:

$$J = \sum_{i=1}^K \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

其中, $\boldsymbol{\mu}_i$ 是第 i 个簇的中心。局限性包括:

- 需要预先指定簇数 K ;
- 对初始簇中心敏感, 可能导致局部最优解;
- 假设簇的形状为球形, 无法处理复杂分布的数据;
- 对噪声和异常值较为敏感。

7. 在 DBSCAN 算法中:

- 核心点: 一个点的 ϵ -邻域内至少包含 $MinPts$ 个点, 则该点为核心点。
- 边界点: 一个点的 ϵ -邻域内的点数少于 $MinPts$, 但它位于某个核心点的 ϵ -邻域内, 则该点为边界点。
- 噪声点: 既不是核心点也不是边界点的点, 被视为噪声。

8. 模糊 C 均值 (FCM) 与 K-Means 的主要区别在于:

- K-Means 是一种硬聚类方法, 每个数据点只能属于一个簇; 而 FCM 是一种软聚类方法, 允许数据点同时属于多个簇, 并通过隶属度表示归属程度。
- FCM 的目标函数引入了隶属度权重, 公式为:

$$J = \sum_{i=1}^K \sum_{j=1}^N u_{ij}^m \|\mathbf{x}_j - \boldsymbol{\mu}_i\|^2$$

其中, u_{ij} 是数据点 $\mathbf{x}_j$ 对簇 i 的隶属度, $m > 1$ 是模糊指数。

9. 层次聚类的优点:

- 能够揭示数据的层次化结构;
- 无需预先指定簇数;
- 适用于小规模数据集。

层次聚类的缺点:

- 计算复杂度较高, 不适合大规模数据集;
- 对噪声和异常值较为敏感;
- 分裂或合并操作不可逆, 可能导致次优解。

10. 选择合适的聚类算法需要考虑以下因素:

- 数据分布特性: 如果数据分布复杂 (如非凸形簇), 可以选择密度聚类 (如 DBSCAN); 如果数据分布清晰且簇数已知, 可以选择原型聚类 (如 K-Means)。
- 数据规模: 对于大规模数据集, 优先选择计算效率高的算法 (如 Mini-Batch K-Means)。
- 是否需要软分配: 如果需要软分配, 可以选择模糊 C 均值 (FCM)。
- 是否需要层次化结构: 如果需要揭示数据的层次化关系, 可以选择层次聚类。

1.7.2 计算题

1. 给定两个二维点 $\mathbf{x} = (1, 2)$ 和 $\mathbf{y} = (4, 6)$, 计算它们的欧氏距离和曼哈顿距离。
2. 给定两个向量 $\mathbf{x} = (3, 4)$ 和 $\mathbf{y} = (-1, 2)$, 计算它们的余弦相似度。
3. 给定两个集合 $A = \{1, 2, 3\}$ 和 $B = \{2, 3, 4\}$, 计算它们的杰卡德距离。

4. 使用 K-Means 算法对以下二维数据点进行聚类, 假设簇数 $K = 2$, 初始簇中心为 $(1, 1)$ 和 $(8, 8)$ 。写出每一步的簇分配和簇中心更新过程。数据点: $(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)$
5. 给定以下数据点, 使用 DBSCAN 算法进行聚类, 参数设置为 $\epsilon = 2$, $MinPts = 2$ 。写出每个点的分类结果。数据点: $(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)$
6. 给定以下隶属度矩阵, 计算每个数据点的簇归属 (取最大隶属度对应的簇)。隶属度矩阵:

$$\begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \\ 0.6 & 0.4 \end{bmatrix}$$

答案:

1. 欧氏距离和曼哈顿距离的计算如下:

- 欧氏距离:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(4-1)^2 + (6-2)^2} = \sqrt{9+16} = 5$$

- 曼哈顿距离:

$$d(\mathbf{x}, \mathbf{y}) = |4-1| + |6-2| = 3+4 = 7$$

2. 余弦相似度的计算如下:

$$\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

其中:

- 内积: $\mathbf{x} \cdot \mathbf{y} = 3 \cdot (-1) + 4 \cdot 2 = -3 + 8 = 5$
- 模长: $\|\mathbf{x}\| = \sqrt{3^2 + 4^2} = 5$, $\|\mathbf{y}\| = \sqrt{(-1)^2 + 2^2} = \sqrt{5}$

因此:

$$\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{5}{5 \cdot \sqrt{5}} = \frac{1}{\sqrt{5}}$$

3. 杰卡德距离的计算如下:

$$d(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

其中:

- $A \cap B = \{2, 3\}$, 交集大小为 2。
- $A \cup B = \{1, 2, 3, 4\}$, 并集大小为 4。

因此:

$$d(A, B) = 1 - \frac{2}{4} = 0.5$$

4. 初始簇中心: $\mu_1 = (1, 1)$, $\mu_2 = (8, 8)$ 。

- 第一步: 分配数据点到最近的簇中心
 - 簇 1: $(1, 1), (1, 2), (2, 1)$
 - 簇 2: $(8, 8), (8, 9), (9, 8)$

- 第二步：更新簇中心

$$\mu_1 = \frac{(1,1) + (1,2) + (2,1)}{3} = (1.33, 1.33)$$

$$\mu_2 = \frac{(8,8) + (8,9) + (9,8)}{3} = (8.33, 8.33)$$

- 第三步：重新分配数据点，簇中心不再变化，算法收敛。最终结果：

– 簇 1: (1, 1), (1, 2), (2, 1)

– 簇 2: (8, 8), (8, 9), (9, 8)

5. • 核心点: (1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)

- 边界点: 无

- 噪声点: 无最终结果:

– 簇 1: (1, 1), (1, 2), (2, 1)

– 簇 2: (8, 8), (8, 9), (9, 8)

6. • 数据点 1: 簇 1 (隶属度 0.8)
• 数据点 2: 簇 2 (隶属度 0.7)
• 数据点 3: 簇 1 (隶属度 0.6)

1.7.3 编程题

1. 编写 Python 代码实现 K-Means 算法，并将其应用于以下二维数据点，簇数 $K = 2$ 。数据点: (1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)
2. 使用 DBSCAN 算法对以下数据点进行聚类，参数设置为 $\epsilon = 2$, $MinPts = 2$ 。数据点: (1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)
3. 使用凝聚层次聚类算法对以下数据点进行聚类，并绘制树状图。数据点: (0.1, 0.2), (0.15, 0.25), (0.2, 0.3), (0.8, 0.9), (0.85, 0.95), (0.9, 1.0)

答案:

1. Python 代码实现 K-Means 算法:

```

1  import numpy as np
2
3  # 数据集
4  data = np.array([[1, 1], [1, 2], [2, 1],
5                  [8, 8], [8, 9], [9, 8]])
6
7  # 初始化簇中心
8  centroids = np.array([[1, 1], [8, 8]])
9
10 # K-Means 算法
11 for _ in range(10): # 最大迭代次数
12     # 分配数据点到最近的簇
13     distances = np.linalg.norm(data[:, np.newaxis] - centroids, axis=2)
14     labels = np.argmin(distances, axis=1)
15
16     # 更新簇中心
17     new_centroids = np.array([data[labels == i].mean(axis=0) for i in range(2)])

```

```

18
19     # 检查是否收敛
20     if np.all(centroids == new_centroids):
21         break
22     centroids = new_centroids
23
24     print("簇分配: ", labels)
25     print("簇中心: ", centroids)
26

```

2. Python 代码实现 DBSCAN 算法:

```

1     from sklearn.cluster import DBSCAN
2     import numpy as np
3
4     # 数据集
5     data = np.array([[1, 1], [1, 2], [2, 1],
6                      [8, 8], [8, 9], [9, 8]])
7
8     # 应用DBSCAN算法
9     dbscan = DBSCAN(eps=2, min_samples=2)
10    labels = dbscan.fit_predict(data)
11
12    print("簇分配: ", labels)
13

```

3. Python 代码实现凝聚层次聚类并绘制树状图:

```

1     from scipy.cluster.hierarchy import dendrogram, linkage
2     import matplotlib.pyplot as plt
3
4     # 数据集
5     data = [[0.1, 0.2], [0.15, 0.25], [0.2, 0.3],
6            [0.8, 0.9], [0.85, 0.95], [0.9, 1.0]]
7
8     # 计算层次聚类的链接矩阵
9     Z = linkage(data, method='average')
10
11    # 绘制树状图
12    plt.figure(figsize=(10, 5))
13    dendrogram(Z)
14    plt.title("Hierarchical Clustering Dendrogram")
15    plt.xlabel("Data Points")
16    plt.ylabel("Distance")
17    plt.show()
18

```

1.8 数据降维-章节习题

1.8.1 简答题

1. 解释核化主成分分析 (Kernel PCA) 与标准 PCA 的主要区别是什么?
2. 为什么在应用核化 PCA 时需要选择适当的核函数?

答案:

1. 标准 PCA 是一种线性降维技术，它通过寻找数据中的最大方差方向来进行降维；而核化 PCA 则使用核技巧将原始数据映射到一个更高维度的空间，在这个新的空间里进行 PCA，然后将结果映射回原始特征空间。这使得核化 PCA 能够捕捉到非线性的结构，这是标准 PCA 所不能做到的。
2. 不同的核函数适用于不同类型的数据分布。选择合适的核函数可以帮助模型更好地捕捉数据内部的结构。例如，径向基函数（RBF）对于处理复杂的非线性模式非常有效；而多项式核可能更适合于那些可以通过高次项来表达关系的数据集。因此，正确选择核函数是确保降维效果良好的关键步骤之一。

1.8.2 计算题

1. 假设我们有一个二维数据集，其中每个样本点都位于单位圆周上。请说明为什么在这种情况下，标准 PCA 无法有效地进行降维，而核化 PCA 可以？

答案：

1. 对于位于单位圆周上的数据点来说，它们之间的差异主要体现在角度而非距离上。因为标准 PCA 寻找的是最大化方差的方向，而在这种情况下，所有点离原点的距离相等，所以标准 PCA 不会提供有意义的结果。相反，如果使用了适当的核函数（如 RBF 核），核化 PCA 可以在高维空间中找到能够区分不同角度位置的新坐标轴，从而实现有效的降维。

1.8.3 编程题

1. 使用 Python 和 scikit-learn 库，编写一段代码来加载 MNIST 手写数字数据集，并应用核化 PCA 对其进行降维。绘制出前两个主成分的散点图，并解释你观察到了什么？

答案：

1.

```
1  # 导入必要的库
2  from sklearn.decomposition import KernelPCA
3  from sklearn.datasets import fetch_openml
4  import matplotlib.pyplot as plt
5
6  # 加载MNIST数据集
7  mnist = fetch_openml('mnist_784', version=1)
8  X, y = mnist["data"], mnist["target"]
9
10 # 初始化KernelPCA模型，这里选择RBF核
11 kpca = KernelPCA(n_components=2, kernel="rbf", gamma=0.0001)
12
13 # 拟合模型并转换数据
14 X_kpca = kpca.fit_transform(X)
15
16 # 绘制散点图
17 plt.figure(figsize=(10,8))
18 for digit in range(10):
19     plt.scatter(X_kpca[y == str(digit), 0], X_kpca[y == str(digit), 1], label=str(digit))
20
21 plt.title("Kernel PCA of MNIST dataset")
22 plt.xlabel("Principal component 1")
23 plt.ylabel("Principal component 2")
```

```
24 plt.legend()  
25 plt.show()  
26
```

运行上述代码后，会得到一个展示 MNIST 数据集中不同数字标签的二维散点图。从图中可以看出，尽管有些类别的簇之间存在一定的重叠，但大多数数字类别还是能够在二维空间中被较好地区分开来。这是因为核化 PCA 成功地找到了一个新的特征空间，在这个空间中，不同数字之间的差异变得更加明显。这证明了核化 PCA 对于处理复杂数据集的有效性，尤其是在数据具有非线性结构的情况下。

1.9 集成学习-章节练习

1.9.1 简答题

1. 什么是集成学习？它与单一模型的主要区别是什么？
2. Bagging 和 Boosting 的主要区别是什么？
3. Stacking 的元模型有什么作用？它的输入特征是如何生成的？

答案：

1. 集成学习是一种通过结合多个基学习器来提升模型性能的方法。其核心思想是“集体智慧优于个体智慧”。与单一模型相比，集成学习能够有效降低模型的偏差或方差，从而提高预测的准确性与鲁棒性。
2. Bagging 通过对训练集进行多次随机采样并训练多个基模型，然后将它们的预测结果进行平均或投票，主要用于降低模型的方差。Boosting 则通过迭代训练多个基模型，每个模型专注于修正前一个模型的错误，主要用于降低模型的偏差。
3. Stacking 的元模型的作用是对多个基模型的预测结果进行整合，从而生成更优的预测。元模型的输入特征通常是基模型在验证集上的预测结果（即交叉验证生成的预测值）。

1.9.2 计算题

1. 假设我们使用 Bagging 方法构建了一个随机森林模型，其中有 100 棵决策树。每棵树的分类准确率为 70%。如果采用多数投票的方式进行集成，求最终模型的分类准确率。（假设每棵树的预测相互独立）
2. 假设我们使用 AdaBoost 方法训练了一个二分类模型，初始样本权重为 $w_1 = w_2 = \dots = w_n = \frac{1}{n}$ 。第一轮训练后，分类器的错误率为 $\epsilon = 0.3$ 。求该分类器的权重 α 。

答案：

1. 根据二项分布公式，最终模型的分类准确率可以通过以下公式计算：

$$P(\text{正确}) = \sum_{k=51}^{100} \binom{100}{k} (0.7)^k (0.3)^{100-k}$$

使用近似计算，当树的数量较大时，可以认为最终准确率接近单棵树的准确率，约为 70%。

2. AdaBoost 中分类器的权重公式为:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right)$$

代入 $\epsilon = 0.3$:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - 0.3}{0.3} \right) = \frac{1}{2} \ln \left(\frac{0.7}{0.3} \right) \approx 0.4236$$

1.9.3 编程题

1. 使用 scikit-learn 实现一个基于随机森林的分类模型，并完成以下任务：

- 加载鸢尾花数据集并划分为训练集和测试集。
- 构建随机森林模型并训练。
- 计算测试集的分类准确率，并绘制特征重要性图。

2. 使用 scikit-learn 实现一个基于梯度提升树的回归模型，并完成以下任务：

- 加载加利福尼亚房价数据集并划分为训练集和测试集。
- 构建梯度提升树模型并训练。
- 计算测试集的均方误差（MSE）和决定系数（ R^2 ）。

答案：

1.

```

1  from sklearn.datasets import load_iris
2  from sklearn.ensemble import RandomForestClassifier
3  from sklearn.model_selection import train_test_split
4  import matplotlib.pyplot as plt
5  import numpy as np
6
7  # 加载数据集
8  iris = load_iris()
9  X, y = iris.data, iris.target
10
11 # 划分训练集和测试集
12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
13
14 # 构建随机森林模型
15 clf = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
16 clf.fit(X_train, y_train)
17
18 # 测试集预测
19 accuracy = clf.score(X_test, y_test)
20 print("测试集准确率:", accuracy)
21
22 # 绘制特征重要性图
23 feature_importances = clf.feature_importances_
24 plt.barh(range(len(feature_importances)), feature_importances, align='center')
25 plt.yticks(np.arange(len(iris.feature_names)), iris.feature_names)
26 plt.xlabel("特征重要性")
27 plt.title("随机森林特征重要性")
28 plt.show()
29

```

2.

```
1  from sklearn.datasets import fetch_california_housing
2  from sklearn.ensemble import GradientBoostingRegressor
3  from sklearn.model_selection import train_test_split
4  from sklearn.metrics import mean_squared_error, r2_score
5
6  # 加载数据集
7  california = fetch_california_housing()
8  X, y = california.data, california.target
9
10 # 划分训练集和测试集
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
12
13 # 构建梯度提升树模型
14 reg = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=3,
15                                random_state=42)
16 reg.fit(X_train, y_train)
17
18 # 测试集预测
19 y_pred = reg.predict(X_test)
20 mse = mean_squared_error(y_test, y_pred)
21 r2 = r2_score(y_test, y_pred)
22 print("测试集均方误差 (MSE):", mse)
23 print("测试集决定系数 (R^2):", r2)
```

1.10 特征选择-章节练习

为了巩固本章所学内容，我们设计了以下练习题，包括简答题、计算题和编程题。每种题型都旨在帮助读者深入理解特征选择的核心概念及其实际应用。

1.10.1 简答题

1. 什么是特征选择？它在机器学习中的作用是什么？
2. 过滤法、包裹法和嵌入法的主要区别是什么？请分别列举一种常用的方法。
3. 为什么 L1 正则化能够实现特征选择？它与 L2 正则化的主要区别是什么？

答案：

1. 特征选择是从原始特征集中挑选出对模型性能最有贡献的特征子集的过程。其作用是减少冗余特征、降低模型复杂度、提高模型的泛化能力和训练效率。
2. 过滤法基于统计学指标独立于模型进行特征选择（如皮尔逊相关系数）；包裹法通过训练模型动态评估特征子集的性能（如递归特征消除 RFE）；嵌入法将特征选择集成到模型训练过程中（如 L1 正则化）。三者的区别主要在于是否依赖模型以及计算效率。
3. L1 正则化通过对模型参数施加稀疏性约束，将不重要的特征权重置为零，从而实现特征选择。而 L2 正则化通过限制参数的平方和，倾向于使所有特征的权重较小但非零，因此无法实现特征选择。

1.10.2 计算题

1. 给定以下数据集，计算特征 x_1 和目标变量 y 的皮尔逊相关系数：

表 1.5: 示例数据集

样本编号	x_1	y
1	2	3
2	4	5
3	6	7
4	8	9

2. 假设某随机森林模型中，特征 f_1 在两棵树中的信息增益分别为 0.3 和 0.5，特征 f_2 的信息增益分别为 0.2 和 0.4。请计算两个特征的平均重要性，并判断哪个特征更重要。

答案：

1. 首先计算均值：

$$\bar{x}_1 = \frac{2+4+6+8}{4} = 5, \quad \bar{y} = \frac{3+5+7+9}{4} = 6$$

然后计算协方差和标准差：

$$\text{Cov}(x_1, y) = \frac{(2-5)(3-6) + (4-5)(5-6) + (6-5)(7-6) + (8-5)(9-6)}{4} = 5$$

$$\sigma_{x_1} = \sqrt{\frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4}} = 2.236, \quad \sigma_y = 2.236$$

最后计算皮尔逊相关系数：

$$r = \frac{\text{Cov}(x_1, y)}{\sigma_{x_1} \cdot \sigma_y} = \frac{5}{2.236 \times 2.236} \approx 1$$

因此， x_1 和 y 的皮尔逊相关系数为 1。

2. 计算平均重要性：

$$\text{Importance}(f_1) = \frac{0.3+0.5}{2} = 0.4, \quad \text{Importance}(f_2) = \frac{0.2+0.4}{2} = 0.3$$

因此，特征 f_1 更重要。

1.10.3 编程题

根据数据集提供相应的代码实现。

数据集

1. 数据集 1（用于互信息计算）：

```
1 import numpy as np
2
3 # 示例数据集
4 X = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
5 y = np.array([0, 1, 0, 1])
6
```

2. 数据集 2（用于递归特征消除）：

```

1  import pandas as pd
2
3  # 示例数据集
4  data = pd.DataFrame({
5      'f1': [1, 2, 3, 4],
6      'f2': [4, 3, 2, 1],
7      'f3': [1, 1, 1, 1],
8      'target': [0, 1, 0, 1]
9  })
10 X = data[['f1', 'f2', 'f3']]
11 y = data['target']
12

```

题目

1. 使用 Scikit-learn 中的 `mutual_info_classif` 函数，计算数据集 1 中每个特征与目标变量之间的互信息，并筛选出互信息大于 0.2 的特征。
2. 使用递归特征消除（RFE）方法，从数据集 2 中选择最重要的 2 个特征。

答案

1. 编写代码如下：

```

1  from sklearn.feature_selection import mutual_info_classif
2
3  mi_scores = mutual_info_classif(X, y)
4  selected_features = [i for i, score in enumerate(mi_scores) if score > 0.2]
5  print("Selected Features:", selected_features)
6

```

输出结果可能为空，因为示例数据集的互信息值较低。

2. 编写代码如下：

```

1  from sklearn.feature_selection import RFE
2  from sklearn.linear_model import LogisticRegression
3
4  model = LogisticRegression()
5  rfe = RFE(estimator=model, n_features_to_select=2)
6  rfe.fit(X, y)
7  selected_features = X.columns[rfe.support_]
8  print("Selected Features:", selected_features)
9

```

输出结果为：

```

1  Selected Features: Index(['f1', 'f2'], dtype='object')
2

```